



Цифровые технологии управления
транспортными процессами

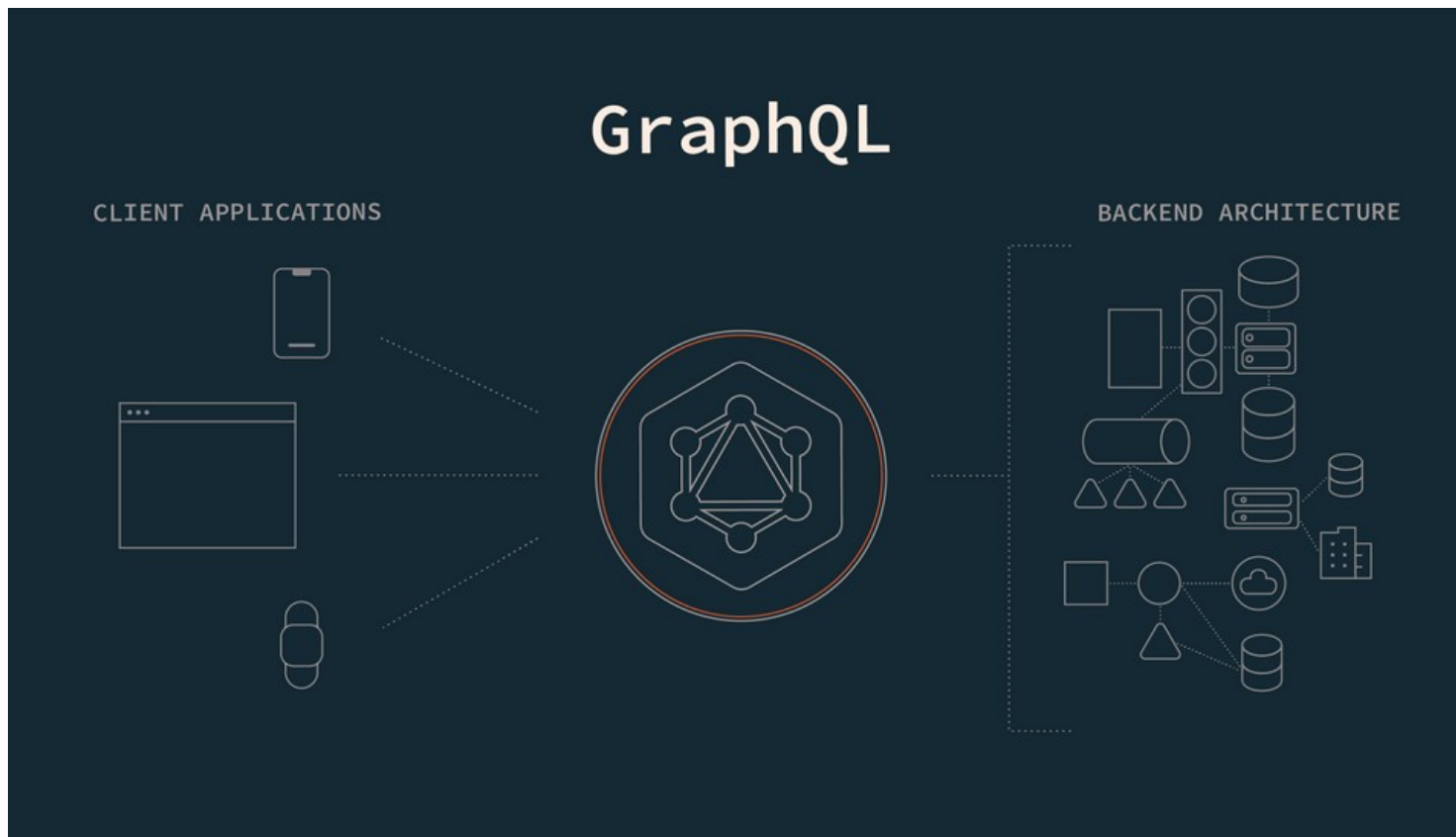
Сервис-ориентированное
программирование

GraphQL

GraphQL+Java



GraphQL



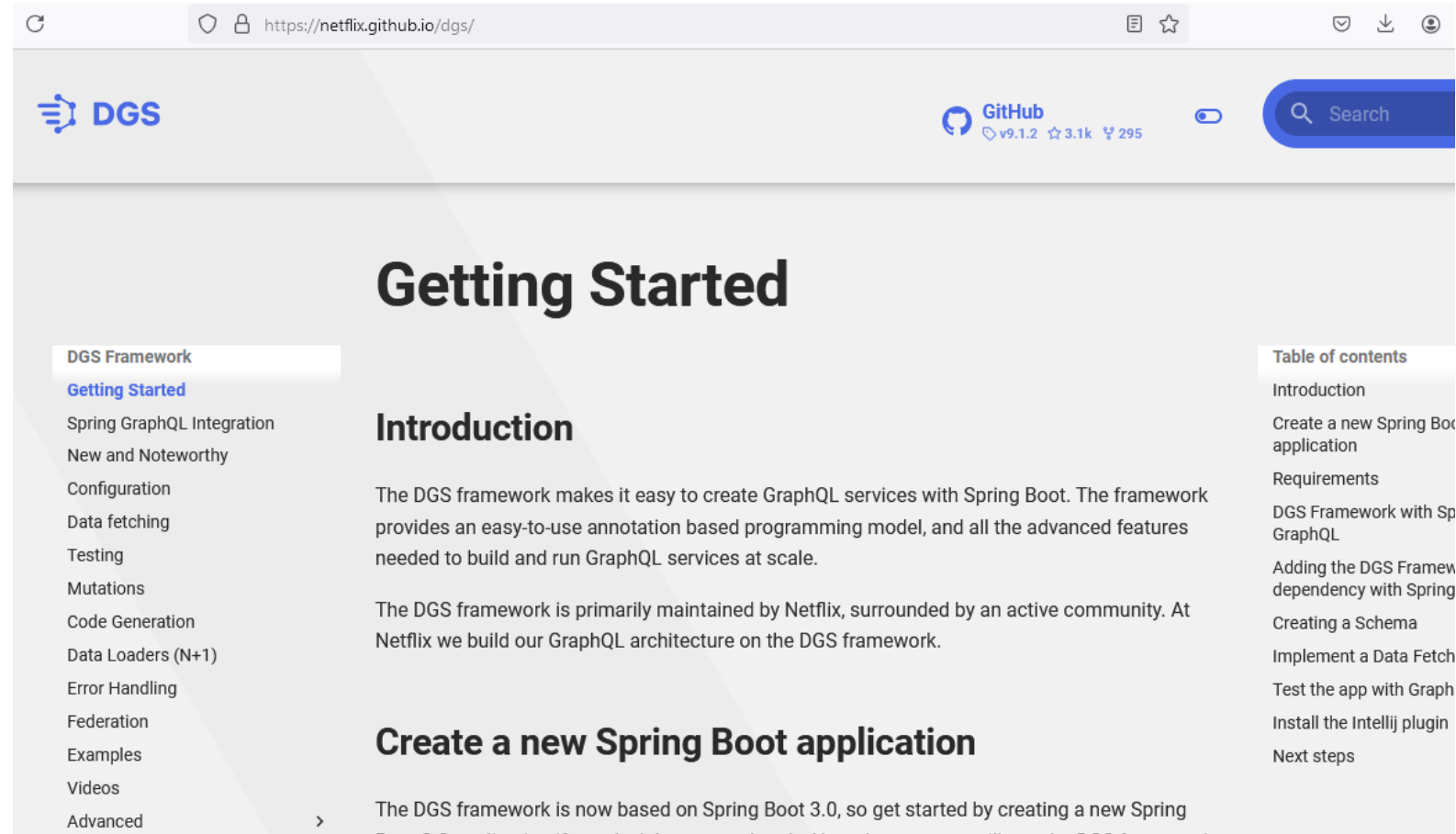


Фреймворк для GraphQL

- GraphQL является спецификацией, по сути языком запросов для API.
- Для внедрения GraphQL на сервере мы обычно используем фреймворки.
- В рамках нашей лабораторной работы мы опробуем Domain Graph Service, фреймворк, созданный командой Netflix, который позволяет использовать GraphQL в приложении Spring Boot подключив всего несколько зависимостей.



Фреймворк для GraphQL



The screenshot shows the 'Getting Started' page for the DGS (Data GraphQL Service) framework. The page is titled 'Getting Started' and features a sidebar with a table of contents. The main content area includes an 'Introduction' section and a 'Create a new Spring Boot application' section. The sidebar on the left lists various topics under the 'DGS Framework' heading, with 'Getting Started' currently selected. The sidebar on the right provides a 'Table of contents' for the 'Getting Started' guide, listing topics from 'Introduction' to 'Next steps'.

Getting Started

Table of contents

- Introduction
- Create a new Spring Boot application
- Requirements
- DGS Framework with Spring GraphQL
- Adding the DGS Framework dependency with Spring
- Creating a Schema
- Implement a Data Fetcher
- Test the app with GraphQL
- Install the IntelliJ plugin
- Next steps

Introduction

The DGS framework makes it easy to create GraphQL services with Spring Boot. The framework provides an easy-to-use annotation based programming model, and all the advanced features needed to build and run GraphQL services at scale.

The DGS framework is primarily maintained by Netflix, surrounded by an active community. At Netflix we build our GraphQL architecture on the DGS framework.

Create a new Spring Boot application

The DGS framework is now based on Spring Boot 3.0, so get started by creating a new Spring Boot 3.0 application if you don't have one already. Note that you can still use the DGS framework...



Фреймворк для GraphQL

- Разработчиками сделан starter, который позволит просто создать проект

The screenshot shows the Spring Initializr web application interface. On the left, there is a sidebar with a hamburger menu icon and a clock icon. The main content area is titled "spring initializr". It contains several sections for configuring a new project:

- Project:** Includes radio buttons for "Gradle - Groovy", "Gradle - Kotlin", "Maven" (selected), and "Language" options: "Java" (selected), "Kotlin", and "Groovy".
- Spring Boot:** Includes radio buttons for versions: "3.4.0 (SNAPSHOT)", "3.4.0 (M3)", "3.3.5 (SNAPSHOT)", "3.3.4" (selected), "3.2.11 (SNAPSHOT)", and "3.2.10".
- Project Metadata:** Includes input fields for "Group" (edu.rut-miit), "Artifact" (graphql-demo), "Name" (graphql-demo), "Description" (GraphQL Demo project for Spring Boot), and "Package name" (edu.rut-miit.graphql-demo).
- Packaging:** Includes radio buttons for "Jar" (selected) and "War".
- Java:** Includes radio buttons for versions: "23", "21" (selected), and "17".
- Dependencies:** Includes a button "ADD DEPENDENCIES... CTRL + B" and two dependency cards: "Spring Web" (WEB) and "Netflix DGS" (WEB).



Фреймворк для GraphQL

- Если хотите добавить стартер в свой проект

```
<dependency>
  <groupId>com.netflix.graphql.dgs</groupId>
  <artifactId>graphql-dgs-spring-graphql-starter</artifactId>
</dependency>
<dependency>
  <groupId>com.netflix.graphql.dgs</groupId>
  <artifactId>graphql-dgs-spring-graphql-starter-test</arti-
factId>
  <scope>test</scope>
</dependency>
```



Scheme First

- Domain Graph Service построен на подходе, когда в начале мы описываем контракт, то есть типы данных, которые могут быть отданы клиентской стороне
- Для этого данный фреймворк проверяет директорию проекта:

`src/main/resources/
schema`

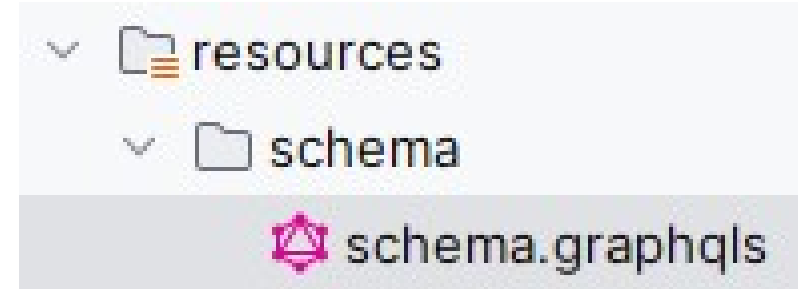
- Давайте в данной директории создадим файл - `schema.graphqls`



schema.graphqls

```
type Query {  
  shows(titleFilter: String):  
  [Show]  
}  
  
type Show {  
  title: String  
  releaseYear: Int  
}
```

- Эта схема позволяет запрашивать список шоу, при необходимости фильтруя их по названию.





Data Fetcher

- Data Fetcher отвечают за возврат данных для запроса, они же применяются и в Spring GraphQL
 - Для концепции GraphQL они считаются resolver
- Создадим директорию datafetchers и класс - ShowsDataFetcher и record - Show

```
record Show(String title, int re-  
leaseYear) {}
```



Data Fetcher

```
...  
@DgsComponent  
public class ShowsDataFetcher {  
    private List<Show> shows = new ArrayList<>(List.of(  
        new Show("Stranger Things", 2016),  
        new Show("Ozark", 2017),  
        new Show("The Crown", 2016),  
        new Show("Dead to Me", 2019),  
        new Show("Orange is the New Black", 2013)  
    ));  
@DgsQuery  
    public List<Show> shows(@InputArgument String titleFilter) {  
        if(titleFilter == null) {  
            return shows;  
        }  
  
        return shows.stream().filter(s ->  
s.title().contains(titleFilter)).collect(Collectors.toList());  
    }  
}
```



Объяснение

- Data Fetcher это resolver а значит он должен решить от куда брать данные, обычно он обращается к сервисам!
- В ваших проектах стоит сделать сервисы, которые могут ходить к репозиториям.
- В демонстрационном примере от DGS используется статический список для демонстрации

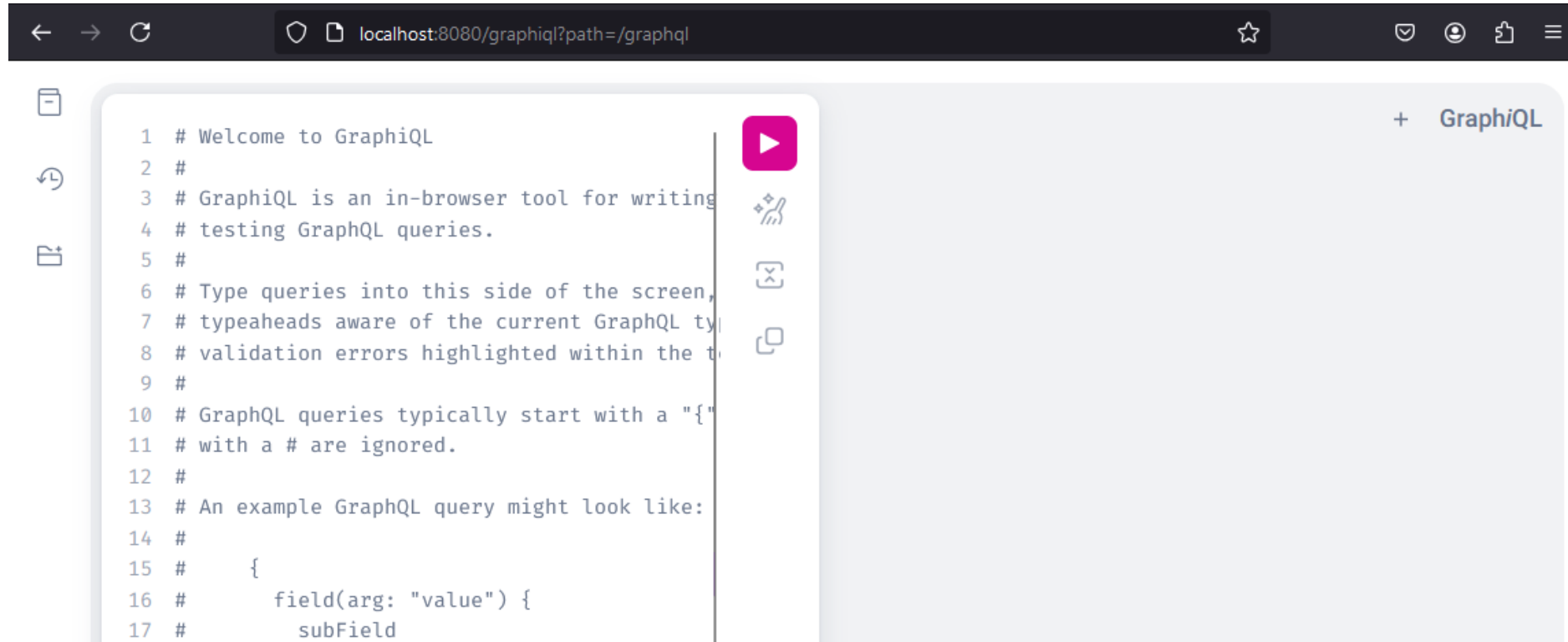


GraphiQL

- Протестировать все запросы можно в среде GraphiQL – интерфейс запросов для тестирования API
- Зачастую данный интерфейс доступен из коробки запускается при старте приложения
- Запустите проект и перейдите по ссылке:
- <http://localhost:8080/graphiql>



GraphiQL





Тестим запросы

- Протестировать все запросы можно в среде GraphiQL – интерфейс запросов для тестирования API
- Зачастую данный интерфейс доступен из коробки запускается при старте приложения
- Запустите проект и перейдите по ссылке:
- <http://localhost:8080/graphiql>



Тестим запросы

```
1 query {
2   shows {
3     title
4     releaseYear
5   }
6 }
7
```

Variables

Headers

1 {}

+ GraphQL

```
{
  "data": {
    "shows": [
      {
        "title": "Stranger Things",
        "releaseYear": 2016
      },
      {
        "title": "Ozark",
        "releaseYear": 2017
      },
      {
        "title": "The Crown",
        "releaseYear": 2016
      },
      {
        "title": "Dead to Me",
        "releaseYear": 2019
      },
      {
        "title": "Orange is the New Black",
        "releaseYear": 2013
      }
    ]
  }
}
```



Добавим мутацию!

- Добавим возможность добавлять новые шоу, для этого необходимо сделать следующее:
 - Метод в DataFetcher

```
@DgsMutation  
public Show addShow(@InputArgument SubmittedShow show) {  
    Show newShow = new  
    Show(show.title(), show.releaseYear());  
    shows.add(newShow);  
    return newShow;  
}
```



Добавим мутацию!

- Добавим возможность добавлять новые шоу, для этого необходимо сделать следующее:
 - Метод в схеме

```
type Mutation {  
  addShow(show: SubmittedShow): Show  
}  
  
input SubmittedShow {  
  title: String!  
  releaseYear: Int!  
}
```



Тестим мутацию!

```
1 mutation {  
2   addShow(show: { title: "Clarkson's Farm", releaseYear: 2021 })  
3     title  
4     releaseYear  
5 }  
6 }
```



```
{  
  "data": {  
    "addShow": {  
      "title": "Clarkson's Farm",  
      "releaseYear": 2021  
    }  
  }  
}
```

+ GraphQL

```
mutation {  
  addShow(show: { title: "Clarkson's Farm", releaseYear:  
2021 }) {  
    title  
    releaseYear  
  }  
}
```



Тестим мутацию!

```
1 query {
2   shows {
3     title
4     releaseYear
5   }
6 }
7
```

Variables Headers

```
1 {}
```

```
{
  "data": {
    "shows": [
      {
        "title": "Stranger Things",
        "releaseYear": 2016
      },
      {
        "title": "Ozark",
        "releaseYear": 2017
      },
      {
        "title": "The Crown",
        "releaseYear": 2016
      },
      {
        "title": "Dead to Me",
        "releaseYear": 2019
      },
      {
        "title": "Orange is the New Black",
        "releaseYear": 2013
      },
      {
        "title": "Clarkson's Farm",
        "releaseYear": 2021
      }
    ]
  }
}
```



Тестим мутацию!

```
1 query {  
2   shows {  
3     title  
4   }  
5 }  
6
```

Variables Headers



```
{  
  "data": {  
    "shows": [  
      {  
        "title": "Stranger Things"  
      },  
      {  
        "title": "Ozark"  
      },  
      {  
        "title": "The Crown"  
      },  
      {  
        "title": "Dead to Me"  
      },  
      {  
        "title": "Orange is the New Black"  
      },  
      {  
        "title": "Clarkson's Farm"  
      }  
    ]  
  }  
}
```



Тестим мутацию!

```
1 query {  
2   shows(titleFilter: "Clarkson's Farm") {  
3     title  
4     releaseYear  
5   }  
6 }  
7
```



```
+ GraphQL  
{  
  "data": {  
    "shows": [  
      {  
        "title": "Clarkson's Farm",  
        "releaseYear": 2021  
      }  
    ]  
  }  
}
```

```
query {  
  shows(titleFilter: "Clarkson's  
Farm") {  
    title  
    releaseYear  
  }  
}
```



Цифровые технологии управления
транспортными процессами

Сервис-ориентированное
программирование

GraphQL

Документация

- <https://netflix.github.io/dgs/>



Будут вопросы? Не стесняйтесь!

